Building a Web-Based Hangman Game with Codex

CSE 4095 Individual Assignment | Development Report | October 29, 2026

Project overview

I built an original web-based Hangman game with an independent Python engine, three difficulty-based word lists, a Flask REST API, and a responsive HTML/CSS/JavaScript interface. The server owns every game decision. During an active game, the browser receives only the masked pattern, guessed letters, incorrect letters, remaining guesses, and status. The answer is returned only after a win or loss.

The implemented request flow is: **browser -> same-origin fetch -> Flask route -> HangmanGame and WordManager -> client-safe JSON -> DOM update**. The application has exactly three game endpoints: `POST /api/games`, `GET /api/games/<game_id>`, and `POST /api/games/<game_id>/guess`.

How I used Codex

I used Codex as a phased coding agent instead of asking for the whole application in one step. First, I supplied the assignment summary, original broad specification, instructor TODO reference, and instructor transcript. I told the agent to treat the last two as references rather than instructions or completed work. Codex created a new unchecked plan and scope record before writing application code.

For each later phase, Codex inspected the real repository state, implemented only that layer, ran focused and full checks, updated the TODO and activity log, and created a local commit. The order was game engine, word manager, Flask API, HTML/CSS presentation, browser JavaScript, reliability testing, browser walkthrough, and Render preparation. I used the reported failures as evidence to decide whether a phase was actually complete.

Implementation and verification

- `HangmanGame` validates one ASCII letter, rejects duplicates, tracks six incorrect guesses, reveals repeated letters, locks completed games, and serializes a safe state.
- `WordManager` loads Easy, Medium, and Hard files relative to the module, rejects invalid data, removes duplicates, and randomly selects only from the requested level.
- Flask stores independent games under unpredictable IDs and strictly validates JSON bodies and status codes.
- The interface supports mouse and physical-keyboard guesses, six SVG drawing stages, stale-request protection, safe `textContent` rendering, win/loss feedback, restart without reload, and mobile/desktop layouts.
- A clean virtual environment passed **58 automated tests**. Node syntax checking and Python compilation passed. A real Gunicorn 23.0.0 process returned HTTP 200 for the page and HTTP 201 for game creation.
- A real local browser walkthrough covered all three levels, click and keyboard input, a six-part loss, a complete win, restart, responsive breakpoints, active-answer secrecy, and network-error recovery.
- The public Render service repeated the critical checks: browser start/guess behavior, 201/200/400/404 endpoint responses, no active `word` field, and terminal answer disclosure only after a real loss.

Problems found and fixed

The work included real failures rather than a fabricated all-green history. The first focused engine command could not import `tests.test_game`; adding `tests/__init__.py` fixed module discovery. A passing API test emitted an unclosed-file `ResourceWarning`; explicitly closing the static response removed it, and the suite then passed with resource warnings treated as errors.

Local server binding was initially denied by the execution sandbox. After scoped localhost permission was granted, I captured authentic 201, 200, 400, and 404 responses. During production testing, an inline `PORT=5056 ... $PORT` command expanded an empty value in zsh. I corrected the README to export `PORT` first, then verified the one-worker Gunicorn command successfully.

Representative prompts I used

> *"Implement ONLY the game-engine phase in TODO.md. In game.py implement an independently testable HangmanGame class that is not coupled to Flask, files, or the browser."*
>
> *"Use ONLY server responses to decide status, pattern, guessed letters, incorrect letters, remaining lives and win/loss; do not store or derive the active answer in JavaScript."*
>
> *"Perform the dedicated testing and reliability phase only... record exact commands, test counts, outcomes, and any unresolved limitations."*

These prompts helped keep each change reviewable and prevented later features from being treated as complete before their tests and evidence existed.

Deployment status and reflection

The public source repository is **https://github.com/armaan-are/cse4095-hangman-web-game** and the live application is **https://cse4095-hangman-web-game.onrender.com**. Render deployed verified commit `57ed38e` on its Free plan using `pip install -r requirements.txt` and one Gunicorn worker. The build succeeded, the health request returned 200, and the service became live. Public checks created, retrieved, guessed, validated bad input, handled an unknown ID, and finished a game; the answer remained private while playing and appeared only after the terminal loss.

AI assistance made it faster to turn a broad specification into small testable phases, but it did not remove the need for judgment. The most valuable practice was requiring direct evidence: a test count, an HTTP response, an observed browser state, or a real commit. The agent also made mistakes, including a brittle test and an incorrect shell example. Running the software exposed those issues. My main lesson is that AI-generated code becomes trustworthy only after I inspect the design boundaries, reproduce the checks, and keep unverified external work visibly incomplete.